

Chapter 24 — Graph Representation

0. What You Will Learn in This Chapter

Graph Representation is the foundation for all graph problems.

Before solving graph DFS, graph BFS, shortest path, topological sort, union find, or minimum spanning tree, you must know how to store the graph.

A graph contains:

- nodes / vertices
- edges / connections

The same graph can be represented in multiple ways:

- adjacency list
- adjacency matrix
- edge list
- implicit graph
- grid graph
- weighted adjacency list
- directed graph representation
- undirected graph representation

By the end of this chapter, you should understand:

- What a graph is
- What vertices and edges are
- Directed vs undirected graphs
- Weighted vs unweighted graphs
- Connected vs disconnected graphs
- Cyclic vs acyclic graphs
- Adjacency list representation
- Adjacency matrix representation
- Edge list representation
- Grid as graph
- Implicit graph representation
- How to build graphs from input
- How to choose the right representation
- How graph representation affects time and space complexity
- Common mistakes in graph construction

Core problems covered in this chapter:

1. Find if Path Exists in Graph
2. Number of Connected Components in an Undirected Graph
3. Clone Graph
4. All Paths From Source to Target
5. Find Center of Star Graph
6. Reorder Routes to Make All Paths Lead to City Zero
7. Keys and Rooms
8. Course Schedule representation
9. Course Schedule II representation
10. Network Delay Time representation
11. Cheapest Flights Within K Stops representation
12. Evaluate Division representation

13. Accounts Merge representation
 14. Grid graph representation
 15. Word Ladder implicit graph representation
-

1. What Is a Graph?

A graph is a collection of nodes connected by edges.

Nodes are also called:

vertices

Edges are also called:

connections

Example:

```
0 -- 1
|   |
2 -- 3
```

Nodes:

0, 1, 2, 3

Edges:

0-1
0-2
1-3
2-3

Graphs are used to model relationships.

Examples:

- cities connected by roads
 - people connected by friendships
 - courses connected by prerequisites
 - computers connected in a network
 - cells connected in a grid
 - words connected by one-letter transformations
-

2. Why Graph Representation Matters

Before applying an algorithm, you need a structure that lets you efficiently answer:

From this node, where can I go next?

This is the most important graph operation.

For node u , we usually need all neighbors:

```
for (int v : graph[u]) {  
    // visit neighbor v  
}
```

If graph representation is bad, traversal becomes slow or complicated.
Good graph representation makes DFS, BFS, Dijkstra, topological sort, and union find easier.

3. Basic Graph Terms

3.1 Vertex / Node

A point in the graph.

Example:

```
0  
1  
2
```

3.2 Edge

A connection between two nodes.

Example:

```
0 -- 1
```

3.3 Neighbor

If there is an edge from u to v , then v is a neighbor of u .

3.4 Degree

Number of edges connected to a node.

For undirected graph:

```
degree = number of neighbors
```

3.5 Path

A sequence of nodes connected by edges.

Example:

```
0 -> 1 -> 3
```

3.6 Cycle

A path that starts and ends at the same node.

Example:

```
0 -> 1 -> 2 -> 0
```

4. Types of Graphs

4.1 Undirected Graph

Edges have no direction.

If u is connected to v , then v is connected to u .

Example:

0 -- 1

Representation:

```
graph[0].push_back(1);  
graph[1].push_back(0);
```

4.2 Directed Graph

Edges have direction.

If $u \rightarrow v$, you can go from u to v , but not necessarily from v to u .

Example:

0 $\rightarrow$ 1

Representation:

```
graph[0].push_back(1);
```

Do not automatically add:

```
graph[1].push_back(0);
```

4.3 Weighted Graph

Edges have cost, distance, or weight.

Example:

0 $\xrightarrow{5}$ 1

Representation:

```
graph[0].push_back({1, 5});  
graph[1].push_back({0, 5});
```

4.4 Unweighted Graph

Edges do not have weights.

Every edge is treated equally.

Representation:

```
graph[u].push_back(v);
```

4.5 Connected Graph

Every node can reach every other node.

Applies mainly to undirected graphs.

4.6 Disconnected Graph

Some nodes cannot reach other nodes.

Example:

0 -- 1 2 -- 3

There are two components:

{0,1}
{2,3}

4.7 DAG

DAG means:

Directed Acyclic Graph

It is a directed graph with no cycle.

Common in:

- course prerequisites
- task scheduling
- dependency graphs
- topological sorting

5. Core Need / Why Learn Graph Representation First?

Graph algorithms depend on representation.

Examples:

DFS / BFS

Need neighbors quickly.

Best representation:

adjacency list

Dense Connectivity Query

Need quickly check if edge exists between u and v.

Best representation:

adjacency matrix

Kruskal MST

Need sorted edges.

Best representation:

edge list

Grid Problems

Graph is implicit.

No need to build adjacency list.

Use directions:

```
vector<pair<int, int>> dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
```

6. Main Graph Representations

There are five major graph representations:

16. adjacency list

- 17. adjacency matrix
 - 18. edge list
 - 19. grid / matrix graph
 - 20. implicit graph
-

7. Representation 1 — Adjacency List

Adjacency list stores for each node all its neighbors.

Example graph:

```
0 -- 1
|   |
2 -- 3
```

Adjacency list:

```
0: 1, 2
1: 0, 3
2: 0, 3
3: 1, 2
```

C++:

```
vector<vector<int>> graph(n);
```

Add undirected edge:

```
graph[u].push_back(v);
graph[v].push_back(u);
```

Add directed edge:

```
graph[u].push_back(v);
```

8. When to Use Adjacency List

Use adjacency list for most graph problems.

It is best when:

- graph is sparse
- need DFS/BFS
- need neighbors of each node
- number of edges is much smaller than $n * n$
- graph can be disconnected
- graph is directed or undirected

Most LeetCode graph problems use adjacency list.

9. Adjacency List Complexity

Let:

n = number of nodes
m = number of edges

Space:

$O(n + m)$

For undirected graph, each edge is stored twice:

u -> v
v -> u

Still considered:

$O(n + m)$

Iterating all neighbors across all nodes costs:

$O(n + m)$

Checking if edge exists between u and v costs:

$O(\text{degree}(u))$

unless using unordered_set.

10. Representation 2 — Adjacency Matrix

Adjacency matrix uses a 2D matrix.

```
vector<vector<int>> matrix(n, vector<int>(n, 0));
```

If edge exists from u to v:

```
matrix[u][v] = 1;
```

For undirected graph:

```
matrix[u][v] = 1;  
matrix[v][u] = 1;
```

Example:

```
0 -- 1  
1 -- 2
```

Matrix:

```
  0 1 2  
0: 0 1 0  
1: 1 0 1  
2: 0 1 0
```

11. When to Use Adjacency Matrix

Use adjacency matrix when:

- graph is dense
- need $O(1)$ edge existence check
- n is small
- problem gives matrix input already
- Floyd-Warshall is needed
- graph is naturally represented as matrix

Examples:

- Is there edge from u to v ?
 - Number of Provinces
 - Floyd-Warshall shortest path
 - graph with $n \leq 100$
-

12. Adjacency Matrix Complexity

Space:

$O(n^2)$

Check if edge exists:

$O(1)$

Iterate neighbors of one node:

$O(n)$

Iterate all possible edges:

$O(n^2)$

This is bad for sparse graphs.

If $n = 100000$, matrix is impossible.

13. Representation 3 — Edge List

Edge list stores all edges directly.

Unweighted:

```
vector<pair<int, int>> edges;
```

Weighted:

```
vector<vector<int>> edges;
```

or:

```
struct Edge {  
    int u;  
    int v;  
    int w;  
};
```

Example:

```
0 -- 1 weight 5
1 -- 2 weight 7
```

Edge list:

```
edges = {
    {0, 1, 5},
    {1, 2, 7}
};
```

14. When to Use Edge List

Use edge list when:

- input is given as list of edges
- need sort edges by weight
- using Kruskal's algorithm
- using Bellman-Ford
- processing all edges directly
- graph traversal is not the main operation

Examples:

- Minimum Spanning Tree with Kruskal
- Bellman-Ford shortest path
- Redundant Connection
- checking all connections

15. Edge List Complexity

Space:

```
O(m)
```

Iterating all edges:

```
O(m)
```

Finding neighbors of a node:

```
O(m)
```

So edge list is not ideal for BFS/DFS unless converted to adjacency list.

16. Representation 4 — Weighted Adjacency List

For weighted graph, each neighbor also stores weight.

C++:

```
vector<vector<pair<int, int>>> graph(n);
```

Add undirected weighted edge:

```
graph[u].push_back({v, w});  
graph[v].push_back({u, w});
```

Add directed weighted edge:

```
graph[u].push_back({v, w});
```

Traversal:

```
for (auto [neighbor, weight] : graph[node]) {  
    // use neighbor and weight  
}
```

Used in:

- Dijkstra
 - Prim
 - Network Delay Time
 - Cheapest Flights
 - minimum cost path
 - weighted road networks
-

17. Representation 5 — Grid as Graph

A grid can be treated as a graph.

Each cell is a node.

Neighbors are adjacent cells.

Example:

```
grid[r][c]
```

Neighbors:

```
(r + 1, c)  
(r - 1, c)  
(r, c + 1)  
(r, c - 1)
```

Directions:

```
vector<pair<int, int>> dirs = {  
    {1, 0},  
    {-1, 0},  
    {0, 1},  
    {0, -1}  
};
```

Boundary check:

```
if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) {  
    continue;  
}
```

18. When to Use Grid Graph Representation

Use grid graph for:

- Number of Islands
- Flood Fill
- Rotting Oranges
- Walls and Gates
- Shortest Path in Binary Matrix
- Word Search
- Pacific Atlantic Water Flow
- Surrounded Regions

Usually, do not build adjacency list explicitly.
Just compute neighbors on the fly.

19. Representation 6 — Implicit Graph

Sometimes nodes and edges are not directly given.
You generate neighbors dynamically.

Example:

Word Ladder.

Node:

word

Edge:

two words differ by one character

Instead of building all edges naively, generate possible transformations.

Example:

for each position:

replace with 'a' to 'z'

Implicit graphs appear in:

- Word Ladder
- Open the Lock
- Sliding Puzzle
- Genetic Mutation
- BFS over states
- puzzle states
- game states

20. Choosing the Right Representation

DFS/BFS on sparse graph	adjacency list
Check edge existence quickly	adjacency matrix
Sort/process all edges	edge list
Weighted shortest path	weighted adjacency list
Grid traversal	grid directions
Generate states dynamically	implicit graph
Course prerequisites	directed adjacency list
Connected components	undirected adjacency list
Kruskal MST	edge list
Floyd-Warshall	adjacency matrix

21. Directed vs Undirected Construction

21.1 Undirected Graph

Input:

```
edges = [[0,1], [1,2]]
```

Build:

```
vector<vector<int>> graph(n);

for (auto& edge : edges) {
    int u = edge[0];
    int v = edge[1];

    graph[u].push_back(v);
    graph[v].push_back(u);
}
```

Because edge works both ways.

21.2 Directed Graph

Input:

```
edges = [[0,1], [1,2]]
```

Means:

0 -> 1

1 -> 2

Build:

```
vector<vector<int>> graph(n);

for (auto& edge : edges) {
    int u = edge[0];
    int v = edge[1];

    graph[u].push_back(v);
}
```

Do not add reverse edge.

22. Weighted Graph Construction

Input:

```
times = [[2,1,1], [2,3,1], [3,4,1]]
```

Each edge:

u, v, weight

Build directed weighted graph:

```
vector<vector<pair<int, int>>> graph(n + 1);

for (auto& edge : times) {
    int u = edge[0];
    int v = edge[1];
    int w = edge[2];

    graph[u].push_back({v, w});
}
```

Use n + 1 if nodes are 1-indexed.

23. 0-Indexed vs 1-Indexed Nodes

This is a major source of bugs.

23.1 0-Indexed

Nodes:

0 to n - 1

Graph size:

```
vector<vector<int>> graph(n);
```

23.2 1-Indexed

Nodes:

1 to n

Graph size:

```
vector<vector<int>> graph(n + 1);
```

Loop:

```
for (int node = 1; node <= n; node++)
```

Common problems with 1-indexed nodes:

- Network Delay Time
- some course/task problems
- city road problems

Always check input constraints.

24. Graph With String Nodes

Sometimes nodes are strings.

Example:

```
"JFK" -> "ATL"
```

Use:

```
unordered_map<string, vector<string>> graph;
```

For weighted string graph:

```
unordered_map<string, vector<pair<string, double>>> graph;
```

Example:

```
graph["a"].push_back({"b", 2.0});  
graph["b"].push_back({"a", 0.5});
```

Used in:

- Evaluate Division
 - Reconstruct Itinerary
 - word transformation problems
-

25. Graph With Non-Continuous Integer Labels

If node labels are not 0..n-1, use map.

Example labels:

100, 500, 900

Option 1:

```
unordered_map<int, vector<int>> graph;
```

Option 2:

Coordinate compression:

100 -> 0

500 -> 1

900 -> 2

Use coordinate compression when:

- need array indexing
- need union find
- many operations
- labels are huge

26. Full Problem Walkthrough 1 — LeetCode 1971: Find if Path Exists in Graph

26.1 Problem Summary

Given undirected graph with n nodes and edges.

Return true if there is a path from source to destination.

26.2 Representation Choice

This is an undirected unweighted graph.

Use adjacency list:

```
vector<vector<int>> graph(n);
```

Add both directions.

26.3 C++ Code

```
class Solution {
public:
    bool validPath(int n,
                  vector<vector<int>>& edges,
                  int source,
                  int destination) {
        vector<vector<int>> graph(n);

        for (auto& edge : edges) {
            int u = edge[0];
```

```

        int v = edge[1];

        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    vector<bool> visited(n, false);
    queue<int> q;

    q.push(source);
    visited[source] = true;

    while (!q.empty()) {
        int node = q.front();
        q.pop();

        if (node == destination) {
            return true;
        }

        for (int neighbor : graph[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }

    return false;
}
};

```

26.4 Complexity

Building graph:

$O(n + m)$

BFS:

$O(n + m)$

Space:

$O(n + m)$

26.5 Common Mistake

For undirected graph, forgetting reverse edge:

```
graph[v].push_back(u);
```

27. Full Problem Walkthrough 2 — Number of Connected Components

27.1 Problem Summary

Given n nodes and undirected edges, return number of connected components.

Example:

$n = 5$

edges = [[0,1], [1,2], [3,4]]

Components:

{0,1,2}

{3,4}

Answer:

2

27.2 Representation Choice

Use undirected adjacency list.

Need run BFS/DFS from every unvisited node.

27.3 C++ Code

```
int countComponents(int n, vector<vector<int>>& edges) {
    vector<vector<int>> graph(n);

    for (auto& edge : edges) {
        int u = edge[0];
        int v = edge[1];

        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    vector<bool> visited(n, false);
    int components = 0;

    for (int i = 0; i < n; i++) {
        if (visited[i]) {
```

```

        continue;
    }

    components++;

    queue<int> q;
    q.push(i);
    visited[i] = true;

    while (!q.empty()) {
        int node = q.front();
        q.pop();

        for (int neighbor : graph[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}

return components;
}

```

27.4 Complexity

Time:

$O(n + m)$

Space:

$O(n + m)$

28. Full Problem Walkthrough 3 — LeetCode 133: Clone Graph

28.1 Problem Summary

Given reference node of connected undirected graph, return deep copy.
Each node has:

```

val
neighbors

```

28.2 Representation Idea

Input graph is already represented as nodes with neighbor lists.
Need map original node to cloned node.

```
unordered_map<Node*, Node*> copied;
```

28.3 C++ Code

```
class Solution {
public:
    Node* cloneGraph(Node* node) {
        if (node == nullptr) {
            return nullptr;
        }

        unordered_map<Node*, Node*> copied;
        queue<Node*> q;

        copied[node] = new Node(node->val);
        q.push(node);

        while (!q.empty()) {
            Node* curr = q.front();
            q.pop();

            for (Node* neighbor : curr->neighbors) {
                if (!copied.count(neighbor)) {
                    copied[neighbor] = new Node(neighbor->val);
                    q.push(neighbor);
                }

                copied[curr]->neighbors.push_back(copied[neighbor]);
            }
        }

        return copied[node];
    }
};
```

28.4 Complexity

Time:

$O(V + E)$

Space:

$O(V)$

28.5 Common Mistakes

- Creating duplicate clone for same original node
- Forgetting null input
- Copying values but not neighbor links
- Infinite loop without visited map

29. Full Problem Walkthrough 4 — LeetCode 797: All Paths From Source to Target

29.1 Problem Summary

Given DAG as adjacency list, return all paths from node 0 to node $n - 1$.
Input is already:

```
vector<vector<int>>> graph
```

where $graph[i]$ is neighbors of node i .

29.2 Representation Choice

Graph is already adjacency list.
Use DFS with path.

29.3 C++ Code

```
class Solution {
public:
    void dfs(vector<vector<int>>& graph,
            int node,
            vector<int>& path,
            vector<vector<int>>& result) {
        int target = graph.size() - 1;

        if (node == target) {
            result.push_back(path);
            return;
        }

        for (int neighbor : graph[node]) {
            path.push_back(neighbor);

            dfs(graph, neighbor, path, result);
        }
    }
};
```

```

        path.pop_back();
    }
}

vector<vector<int>> allPathsSourceTarget(vector<vector<int>>& graph) {
    vector<vector<int>> result;
    vector<int> path = {0};

    dfs(graph, 0, path, result);

    return result;
}
};

```

29.4 Complexity

Can be exponential because number of paths can be exponential.
Representation itself uses:

$O(n + m)$

30. Full Problem Walkthrough 5 — LeetCode 1791: Find Center of Star Graph

30.1 Problem Summary

Given edges of star graph, find center node.
Star graph has one center connected to every other node.

30.2 Representation Choice

No need to build graph.
In a star graph, center appears in both first two edges.

30.3 C++ Code

```

class Solution {
public:
    int findCenter(vector<vector<int>>& edges) {
        int a = edges[0][0];
        int b = edges[0][1];

        int c = edges[1][0];
    }
};

```

```
int d = edges[1][1];

if (a == c || a == d) {
    return a;
}

return b;
}
};
```

30.4 Lesson

Sometimes graph representation is unnecessary.
If structure gives direct shortcut, use it.

31. Full Problem Walkthrough 6 — LeetCode 1466: Reorder Routes to Make All Paths Lead to City Zero

31.1 Problem Summary

There are n cities and directed roads.
Need reorder minimum edges so every city can reach city 0.

31.2 Representation Choice

Need traverse as if undirected, but remember original direction.
For edge:

$u \rightarrow v$

Store:

$u \rightarrow v$ with cost 1
 $v \rightarrow u$ with cost 0

If during traversal from 0 we go along original direction away from 0, that edge must be reversed.

31.3 C++ Code

```
class Solution {
public:
    int minReorder(int n, vector<vector<int>>& connections) {
        vector<vector<pair<int, int>>> graph(n);

        for (auto& edge : connections) {
```

```

    int u = edge[0];
    int v = edge[1];

    graph[u].push_back({v, 1});
    graph[v].push_back({u, 0});
}

vector<bool> visited(n, false);
queue<int> q;

q.push(0);
visited[0] = true;

int changes = 0;

while (!q.empty()) {
    int node = q.front();
    q.pop();

    for (auto [neighbor, cost] : graph[node]) {
        if (visited[neighbor]) {
            continue;
        }

        changes += cost;
        visited[neighbor] = true;
        q.push(neighbor);
    }
}

return changes;
}
};

```

31.4 Representation Lesson

Sometimes edge needs metadata.
 Here neighbor alone is not enough.
 Need:

neighbor + whether edge direction is wrong

32. Full Problem Walkthrough 7 —

LeetCode 841: Keys and Rooms

32.1 Problem Summary

There are rooms 0..n-1.

Each room contains keys to other rooms.

Return true if you can visit all rooms starting from room 0.

32.2 Representation Choice

Input already is adjacency list:

rooms[i] = keys from room i

This is directed graph.

If room i has key to room j, edge:

i -> j

32.3 C++ Code

```
class Solution {
public:
    bool canVisitAllRooms(vector<vector<int>>& rooms) {
        int n = rooms.size();

        vector<bool> visited(n, false);
        stack<int> st;

        st.push(0);
        visited[0] = true;

        while (!st.empty()) {
            int room = st.top();
            st.pop();

            for (int key : rooms[room]) {
                if (!visited[key]) {
                    visited[key] = true;
                    st.push(key);
                }
            }
        }

        for (bool seen : visited) {
```

```
        if (!seen) {  
            return false;  
        }  
    }  
  
    return true;  
}  
};
```

32.4 Complexity

Time:

$O(n + \text{totalKeys})$

Space:

$O(n)$

33. Full Problem Walkthrough 8 — Course Schedule Representation

33.1 Problem Summary

Given courses and prerequisites.

Each prerequisite pair:

[a, b]

means:

to take course a, first take course b

So directed edge:

$b \rightarrow a$

33.2 Representation Choice

Directed graph.

Build:

```
graph[b].push_back(a);  
indegree[a]++;
```

33.3 C++ Graph Build

```
vector<vector<int>>> graph(numCourses);  
vector<int> indegree(numCourses, 0);  
  
for (auto& pre : prerequisites) {
```

```
int course = pre[0];
int prereq = pre[1];

graph[prereq].push_back(course);
indegree[course]++;
}
```

33.4 Common Mistake

Reversing the edge incorrectly.

If pair is:

[course, prerequisite]

then edge is:

prerequisite -> course

not:

course -> prerequisite

34. Full Problem Walkthrough 9 — Network Delay Time Representation

34.1 Problem Summary

Given directed weighted edges:

u, v, w

Signal travels from u to v in time w.

Need shortest time from source to all nodes.

34.2 Representation Choice

Directed weighted adjacency list:

```
vector<vector<pair<int, int>>> graph(n + 1);
```

34.3 C++ Graph Build

```
vector<vector<pair<int, int>>> graph(n + 1);
```

```
for (auto& edge : times) {
    int u = edge[0];
    int v = edge[1];
    int w = edge[2];
```

```
graph[u].push_back({v, w});  
}
```

34.4 Why $n + 1$?

In this problem, nodes are labeled:

1 to n

So index 0 is unused.

35. Full Problem Walkthrough 10 — Evaluate Division Representation

35.1 Problem Summary

Given equations:

$a / b = 2.0$

Need answer division queries.

35.2 Representation Choice

This is a weighted directed graph with string nodes.

Equation:

$a / b = 2.0$

Edges:

$a \rightarrow b$ weight 2.0

$b \rightarrow a$ weight 0.5

35.3 C++ Graph Build

```
unordered_map<string, vector<pair<string, double>>> graph;  
  
for (int i = 0; i < equations.size(); i++) {  
    string a = equations[i][0];  
    string b = equations[i][1];  
    double value = values[i];  
  
    graph[a].push_back({b, value});  
    graph[b].push_back({a, 1.0 / value});  
}
```

35.4 Representation Lesson

Graphs do not need integer nodes.

String-labeled graph is common.

Use:

```
unordered_map<string, vector<pair<string, double>>>
```

36. Full Problem Walkthrough 11 — Accounts Merge Representation

36.1 Problem Summary

Accounts have names and emails.

If two accounts share an email, they belong to same person.

Need merge emails.

36.2 Representation Choice

Emails can be nodes.

If emails are in same account, connect them.

For account:

```
John: a@mail, b@mail, c@mail
```

Add edges:

```
a <-> b
```

```
a <-> c
```

Also store:

```
emailToName[email] = name;
```

36.3 C++ Graph Build

```
unordered_map<string, vector<string>> graph;  
unordered_map<string, string> emailToName;
```

```
for (auto& account : accounts) {  
    string name = account[0];  
    string firstEmail = account[1];  
  
    for (int i = 1; i < account.size(); i++) {  
        string email = account[i];  
  
        emailToName[email] = name;  
  
        graph[firstEmail].push_back(email);  
        graph[email].push_back(firstEmail);  
    }  
}
```

36.4 Representation Lesson

Choose nodes based on what connects.

Here accounts are not the best nodes.

Emails are better nodes because shared emails define connection.

37. Full Problem Walkthrough 12 — Grid Graph Representation

37.1 Problem Summary

Many grid problems ask traversal through cells.

Example:

```
1 1 0
0 1 0
1 0 1
```

Each cell can be treated as node.

37.2 Representation Choice

Do not build graph explicitly.

Use directions.

37.3 C++ Template

```
int rows = grid.size();
int cols = grid[0].size();

vector<pair<int, int>> dirs = {
    {1, 0},
    {-1, 0},
    {0, 1},
    {0, -1}
};

for (auto [dr, dc] : dirs) {
    int nr = r + dr;
    int nc = c + dc;

    if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) {
        continue;
    }
}
```

```
// valid neighbor
}
```

37.4 8-Direction Grid

Some problems allow diagonal movement.

```
vector<pair<int, int>> dirs = {
    {1, 0}, {-1, 0}, {0, 1}, {0, -1},
    {1, 1}, {1, -1}, {-1, 1}, {-1, -1}
};
```

Used in:

- Shortest Path in Binary Matrix

38. Full Problem Walkthrough 13 — Word Ladder Implicit Graph

38.1 Problem Summary

Transform begin word to end word by changing one letter at a time.
Each transformed word must exist in dictionary.

38.2 Representation Choice

Words are nodes.

Edges exist between words that differ by one character.

Naively building all edges is expensive.

Use implicit graph:

For each word, generate neighbors by changing each character.

38.3 Neighbor Generation

```
vector<string> getNeighbors(string word, unordered_set<string>& dict) {
    vector<string> neighbors;

    for (int i = 0; i < word.size(); i++) {
        char original = word[i];

        for (char c = 'a'; c <= 'z'; c++) {
            if (c == original) {
                continue;
            }

            word[i] = c;

            if (dict.count(word)) {
```

```
        neighbors.push_back(word);
    }
}

word[i] = original;
}

return neighbors;
}
```

38.4 Representation Lesson

Not every graph should be prebuilt.

If neighbors can be generated quickly, implicit graph saves memory.

39. Common Graph Build Templates

39.1 Undirected Unweighted

```
vector<vector<int>>> graph(n);

for (auto& edge : edges) {
    int u = edge[0];
    int v = edge[1];

    graph[u].push_back(v);
    graph[v].push_back(u);
}
```

39.2 Directed Unweighted

```
vector<vector<int>>> graph(n);

for (auto& edge : edges) {
    int u = edge[0];
    int v = edge[1];

    graph[u].push_back(v);
}
```

39.3 Undirected Weighted

```
vector<vector<pair<int, int>>> graph(n);

for (auto& edge : edges) {
    int u = edge[0];
    int v = edge[1];
    int w = edge[2];

    graph[u].push_back({v, w});
    graph[v].push_back({u, w});
}
```

39.4 Directed Weighted

```
vector<vector<pair<int, int>>> graph(n);

for (auto& edge : edges) {
    int u = edge[0];
    int v = edge[1];
    int w = edge[2];

    graph[u].push_back({v, w});
}
```

39.5 String Graph

```
unordered_map<string, vector<string>> graph;

for (auto& edge : edges) {
    string u = edge[0];
    string v = edge[1];

    graph[u].push_back(v);
}
```

39.6 Weighted String Graph

```
unordered_map<string, vector<pair<string, double>>> graph;

graph[a].push_back({b, value});
graph[b].push_back({a, 1.0 / value});
```

40. Common Edge Cases for Graph Representation

40.1 No Edges

Graph may have isolated nodes.

Example:

```
n = 5  
edges = []
```

Adjacency list should still have size n.

```
vector<vector<int>> graph(n);
```

40.2 Single Node

Handle:

```
n = 1
```

40.3 Disconnected Graph

If need visit all nodes, loop through every node.

```
for (int i = 0; i < n; i++)
```

Do not only start from node 0 unless problem says graph is connected.

40.4 Duplicate Edges

Some inputs may contain duplicate edges.

If duplicates matter, keep them.

If not, use:

```
unordered_set
```

or sort and unique.

40.5 Self Loops

Edge:

```
u -> u
```

Can affect cycle detection.

40.6 1-Indexed Nodes

Use $n + 1$.

40.7 Directed Edge Direction

Read problem carefully.

Course Schedule has edge:

```
prerequisite -> course
```

40.8 Weighted Edges

Do not lose weight while building graph.

41. Common Mistakes and Wrong Code

Mistake 1 — Forgetting Reverse Edge in Undirected Graph

Wrong:

```
graph[u].push_back(v);
```

Correct:

```
graph[u].push_back(v);  
graph[v].push_back(u);
```

Mistake 2 — Adding Reverse Edge in Directed Graph

Wrong:

```
graph[u].push_back(v);  
graph[v].push_back(u);
```

Correct:

```
graph[u].push_back(v);
```

Mistake 3 — Wrong Graph Size

For 1-indexed nodes:

Wrong:

```
vector<vector<int>> graph(n);
```

Correct:

```
vector<vector<int>> graph(n + 1);
```

Mistake 4 — Ignoring Isolated Nodes

Even if node has no edges, it still exists.

Initialize graph with all nodes.

Mistake 5 — Using Matrix for Huge Sparse Graph

Adjacency matrix can cause memory limit exceeded.

Use adjacency list.

Mistake 6 — Using Edge List for BFS

Edge list makes neighbor lookup slow.

Convert to adjacency list.

42. Interview Explanation Templates

42.1 Adjacency List

I represent the graph as an adjacency list where `graph[u]` stores all neighbors of `u`. This makes BFS or DFS efficient because I can iterate only real edges.

42.2 Undirected Graph

Since the graph is undirected, every edge must be added both ways: `u` to `v` and `v` to `u`.

42.3 Directed Graph

Since edges have direction, I only add the edge from source to destination.

42.4 Weighted Graph

I store each neighbor together with the edge weight using pairs like `{neighbor, weight}`.

42.5 Grid Graph

I treat each cell as a graph node and generate neighbors using direction arrays instead of explicitly building an adjacency list.

42.6 Implicit Graph

Instead of precomputing all edges, I generate valid next states during BFS or DFS. This saves memory and is useful when the state space is large.

43. Complexity Patterns

Adjacency list	$O(n + m)$	$O(\text{degree node})$	$O(\text{degree node})$
Adjacency matrix	$O(n^2)$	$O(n)$	$O(1)$
Edge list	$O(m)$	$O(m)$	$O(m)$
Weighted adjacency list	$O(n + m)$	$O(\text{degree node})$	$O(\text{degree node})$
Grid implicit	$O(1)$ extra	$O(4)$ or $O(8)$	boundary check
String graph map	$O(n + m)$	$O(\text{degree node})$	depends on map/set

44. Time-Boxed Practice Guidance

Basic Graph Representation

Target:

15–25 minutes

Problems:

- Find if Path Exists in Graph
- Keys and Rooms
- Number of Connected Components

Medium Representation Practice

Target:

25–45 minutes

Problems:

- Clone Graph
- All Paths From Source to Target
- Reorder Routes
- Course Schedule graph build
- Evaluate Division graph build

Harder Representation Practice

Target:

45–60 minutes

Problems:

- Accounts Merge
- Word Ladder
- Network Delay Time
- Cheapest Flights Within K Stops
- grid graph problems

45. Expanded Practice Set

45.1 Must-Do Easy

21. Find if Path Exists in Graph
22. Find Center of Star Graph
23. Keys and Rooms
24. Number of Connected Components
25. Flood Fill
26. Number of Islands

45.2 Must-Do Medium

27. Clone Graph
28. All Paths From Source to Target
29. Reorder Routes to Make All Paths Lead to City Zero
30. Course Schedule
31. Course Schedule II
32. Evaluate Division
33. Accounts Merge
34. Network Delay Time
35. Cheapest Flights Within K Stops

45.3 Strong Interview Practice

36. Word Ladder

- 37. Open the Lock
- 38. Rotting Oranges
- 39. Pacific Atlantic Water Flow
- 40. Surrounded Regions
- 41. Redundant Connection
- 42. Min Cost to Connect All Points
- 43. Path With Minimum Effort

45.4 Hard / Stretch

- 44. Word Ladder
 - 45. Word Ladder II
 - 46. Alien Dictionary
 - 47. Critical Connections in a Network
 - 48. Swim in Rising Water
 - 49. Shortest Path Visiting All Nodes
 - 50. Bus Routes
-

46. Suggested Study Order

Step 1 — Basic Adjacency List

Start with:

- 51. Find if Path Exists
- 52. Number of Connected Components
- 53. Keys and Rooms

Goal:

Build and traverse unweighted graphs.

Step 2 — Directed Graphs

Solve:

- 54. Course Schedule representation
- 55. Course Schedule II representation
- 56. All Paths From Source to Target

Goal:

Understand edge direction.

Step 3 — Weighted Graphs

Solve:

- 57. Network Delay Time
- 58. Cheapest Flights Within K Stops
- 59. Evaluate Division

Goal:

Store weights properly.

Step 4 — Graphs With Non-Integer Nodes

Solve:

- 60. Evaluate Division
- 61. Accounts Merge

Goal:

Use maps for string-labeled nodes.

Step 5 — Grid and Implicit Graphs

Solve:

- 62. Number of Islands
- 63. Rotting Oranges
- 64. Word Ladder
- 65. Open the Lock

Goal:

Generate neighbors dynamically.

47. Pattern Decision Table

edges = [[u,v]], undirected	adjacency list with both directions
edges = [[u,v]], directed	adjacency list one direction
edges = [[u,v,w]]	weighted adjacency list
need sort edges by weight	edge list
need O(1) edge lookup	adjacency matrix
nodes are strings	unordered_map graph
grid cells	implicit grid directions
word transformations	implicit graph
prerequisites	directed graph + indegree
equations with ratios	weighted string graph
connected components	undirected adjacency list
shortest weighted path	weighted adjacency list

48. Final Cheat Sheet

Undirected Graph

```
vector<vector<int>> graph(n);
```

```
for (auto& edge : edges) {  
    int u = edge[0];  
    int v = edge[1];
```

```
    graph[u].push_back(v);
```

```
graph[v].push_back(u);  
}
```

Directed Graph

```
vector<vector<int>>> graph(n);  
  
for (auto& edge : edges) {  
    int u = edge[0];  
    int v = edge[1];  
  
    graph[u].push_back(v);  
}
```

Weighted Graph

```
vector<vector<pair<int, int>>>> graph(n);  
  
for (auto& edge : edges) {  
    int u = edge[0];  
    int v = edge[1];  
    int w = edge[2];  
  
    graph[u].push_back({v, w});  
}
```

Grid Directions

```
vector<pair<int, int>> dirs = {  
    {1, 0},  
    {-1, 0},  
    {0, 1},  
    {0, -1}  
};
```

String Graph

```
unordered_map<string, vector<string>>> graph;
```

Weighted String Graph

```
unordered_map<string, vector<pair<string, double>>>> graph;
```

49. What to Master Before Chapter 25

Before moving to Graph DFS, make sure you can:

- Explain what a graph is
 - Know directed vs undirected
 - Know weighted vs unweighted
 - Build adjacency list
 - Build adjacency matrix
 - Build edge list
 - Build weighted adjacency list
 - Handle 0-indexed and 1-indexed nodes
 - Represent string-labeled graphs
 - Represent grid graphs using directions
 - Represent implicit graphs
 - Know when to add reverse edge
 - Know when not to add reverse edge
 - Choose correct representation for DFS/BFS
 - Avoid graph size and indexing mistakes
-

50. Core Takeaway

Graph representation answers the question:

From this node, where can I go next?

Most graph problems become easier once the graph is represented correctly.

Use:

- adjacency list for most DFS/BFS problems
- adjacency matrix for dense graphs or $O(1)$ edge checks
- edge list for edge-processing algorithms
- weighted adjacency list for shortest path problems
- grid directions for matrix problems
- implicit graph generation for state-space problems

The most important decision is:

Is the graph directed or undirected?

Because that decides whether you add:

`graph[v].push_back(u);`

or not.

A wrong graph build usually means the whole solution becomes wrong.

Graph Representation is the foundation for all upcoming graph chapters.
